

Ansible desde Cero: Automatiza Linux y Servidores

Cursos PIT – 10ma Edicion

Instructor: Cristian Adrian Diaz Garcia
Fecha: 18 de agosto de 2026



**UNIVERSIDAD
NACIONAL DE
INGENIERÍA**

CURSOS GRATUITOS
OTI UNI



**TRANSFORMACIÓN
digital**

SESIÓN 2

Primer playbook y despliegue de Nginx

SESIÓN 2

- Qué es un playbook y cómo se estructura en YAML.
- Instalación y configuración automatizada de Nginx.
- Idempotencia: cómo Ansible evita repetir cambios innecesarios.
- Ejecución, validación y corrección de errores básicos en playbooks.

Introducción a YAML

Formatos de Datos: XML, JSON y YAML

```
<Servers>
  <Server>
    <name>Server1</name>
    <owner>John</owner>
    <created>123456</created>
    <status>active</status>
  </Server>
</Servers>
```

XML

- Muy formal, basado en etiquetas.
- Verboso para archivos de configuración.

```
{
  Servers: [
    {
      name: Server1,
      owner: John,
      created: 123456,
      status: active
    }
  ]
}
```

JSON

- Ligero, común en APIs y aplicaciones web.
- Usa llaves, corchetes y comillas con frecuencia.

```
Servers:
-   name: Server1
    owner: John
    created: 123456
    status: active
```

YAML

- Legible para humanos, usado en automatización.
- Usa indentación, listas y pares clavevalor.

Reglas Básicas de YAML

```
# Datos del servidor
servidor: web01
puerto: 80
activo: true
paquetes:
  - nginx
  - curl
```

Ideas Claves

- Ideas clave Los comentarios empiezan con #.
- La estructura depende de la **indentación**.
- Se recomiendan **2 espacios**; no usar tabs.
- Las listas usan guion: - *item*.
- Las variables suelen ser clave: *valor*.

Reglas Básicas de YAML

```
usuario:  
  nombre: ansible  
  sudo: true  
  grupos:  
    - admins  
    - devops  
  contacto: {ciudad: Lima, pais: Peru}
```

Escalares

- Valores simples: texto, números, booleanos o nulos.

Mapas

- Pares *clave: valor*, parecidos a un diccionario.

Secuencias

- Listas ordenadas de elementos.

Mini Actividad: Autobiografía en YAML

```
# Autobiografia en YAML
nombre: "Tu nombre"
contacto: {correo: "tu@correo.com", ciudad: "Lima"}
habilidades:
  - Linux
  - Git
  - Ansible
metas:
  corto_plazo: "Practicar automatizacion"
  largo_plazo: "Administrar servidores"
# Fin del documento
```

Crea un archivo ***autobiografia.yaml*** para practicar estructura e indentación.

Playbooks

De Comandos Ad-Hoc a Playbooks

Comandos ad-hoc

- Ideales para pruebas y tareas rápidas.
- Se ejecutan una vez desde la terminal.

```
luis@luis-UX370UAR:~/Escritorio$ ansible pruebascomandos -m shell -a "echo $HOME"
35.238.41.3 | CHANGED | rc=0 >>
/home/luis
34.71.75.10 | CHANGED | rc=0 >>
/home/luis
luis@luis-UX370UAR:~/Escritorio$ ansible pruebascomandos -m shell -a "echo $HOME & date"
34.71.75.10 | CHANGED | rc=0 >>
/home/luis
Sun Jun 21 19:21:19 UTC 2020
35.238.41.3 | CHANGED | rc=0 >>
/home/luis
Sun Jun 21 19:21:19 UTC 2020
```

```
---
- name: Playbook 1 Name of Playbook
  hosts: webservers 2 HostGroup Name
  become: yes 3 Sudo (or) run as different user setting
  become_user: root
  tasks: 4
  - name: ensure apache is at the latest version
    yum:
      name: httpd
      state: latest
  - name: ensure apache is running
    service:
      name: httpd
      state: started
```

Tasks

Playbooks

- Guardan la automatización en archivos YAML.
- Permiten repetir, revisar y versionar procedimientos.

¿Qué es un Playbook?

Definición

Un playbook es un archivo YAML que indica qué hosts serán gestionados y qué tareas ejecutará Ansible para llevarlos a un estado deseado.

```
---  
- name: Mi primer playbook  
  hosts: web  
  become: true  
  tasks:  
    - name: Verificar conectividad  
      ansible.builtin.ping:
```

legible del play

name

Grupo o servidor
objetivo

hosts

Ejecutar con
privilegios

become

Lista de tareas

task

Estructura Común de un Playbook

```
---  
- name: Preparar servidor web  
  hosts: web  
  become: true  
  tasks:  
    - name: Instalar nginx  
      apt:  
        name: nginx  
        state: present
```

- **name:** Es una etiqueta descriptiva
- **hosts:** Indica el grupo del inventario
- **become:** Permite ejecutar las tareas con privilegios elevados
- **tasks:** Lista de tareas secuenciales
- **apt:** Es el módulo de Ansible específico para gestionar paquetes en sistemas Debian/Ubuntu
- **name:** nginx: Define el nombre del paquete específico que se quiere gestionar
- **state:** Define el estado deseado del paquete.

Anatomía de una Tarea (Task)

Una task (tarea) es la unidad básica de ejecución en un playbook, diseñada para invocar un módulo específico (como apt, service o copy).

name:

- describe qué hace la tarea.

ansible.builtin apt:

- módulo usado para gestionar paquetes.

state:

- declara el estado deseado.

```
- name: Instalar Nginx
  ansible.builtin.apt: name:
    nginx state: present
    update_cache: true
```

*Una tarea = nombre + módulo
+ parámetros*

Flujo de Despliegue de Nginx

instalar

habilitar

publicar

validar

Meta

Dejar un servidor web funcionando con una página personalizada usando un solo comando de Ansible.

Playbook: Instalar y Habilitar Nginx

```
---  
- name: Desplegar Nginx  
  hosts: web  
  become: true  
  tasks:  
    - name: Instalar Nginx  
      ansible.builtin.apt:  
        name: nginx  
        state: present  
        update_cache: true  
    - name: Asegurar que Nginx esté activo  
      ansible.builtin.service:  
        name: nginx  
        state: started  
        enabled: true
```

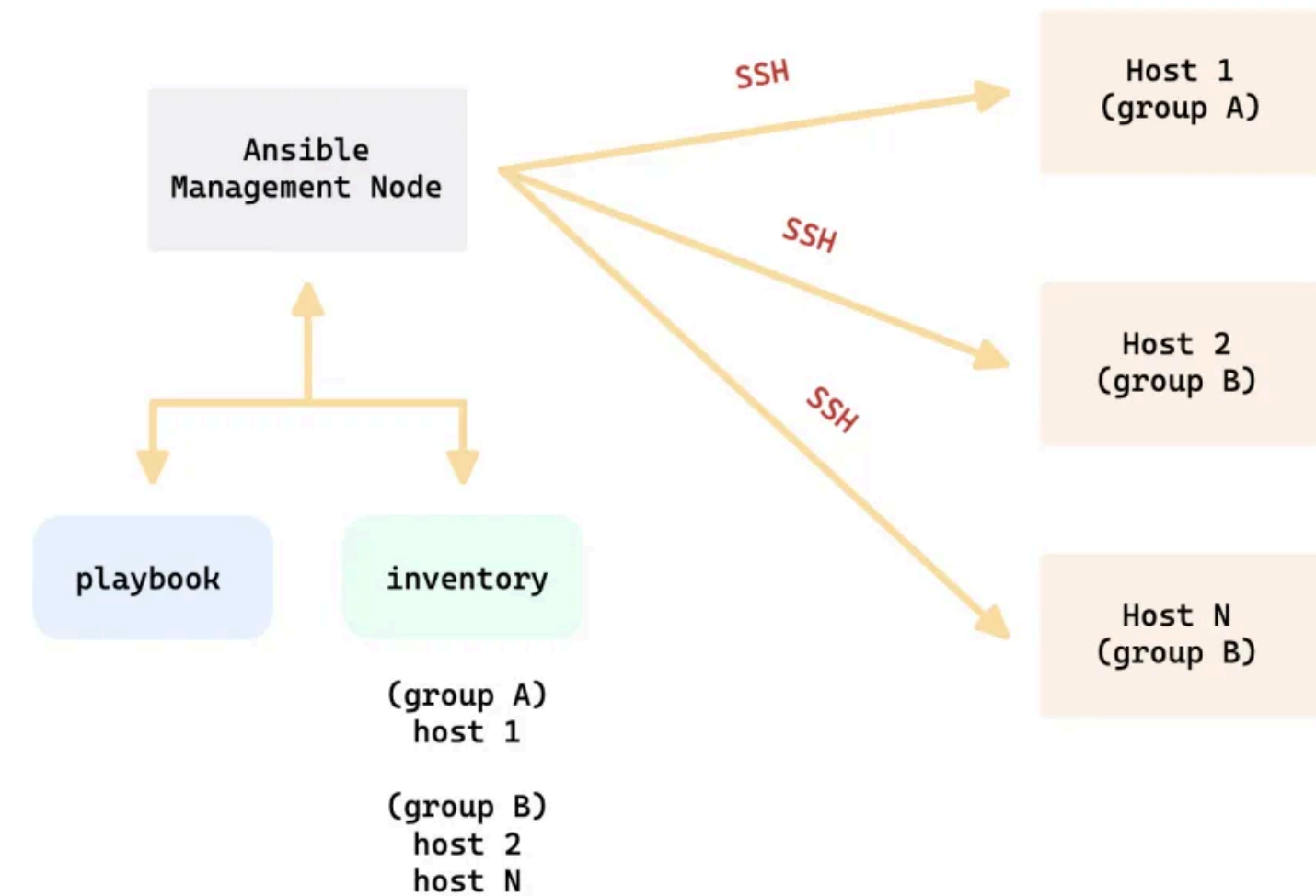
Publicar una Página HTML

```
- name: Publicar página principal
  ansible.builtin.copy:
    dest: /var/www/html/index.html
    content: |
      <h1>Servidor gestionado con Ansible</h1>
      <p>Despliegue automatizado de Nginx.</p>
    owner: www-data
    group: www-data
    mode: '0644'
```


Ejecutar el Playbook

aplica el playbook.

```
ansible-playbook -i inventory.ini nginx.yml
```



Validar el Despliegue

Desde el nodo de control:

Verificamos que el servicio responda y que la página publicada esté disponible.

```
ansible web -i inventory.ini -m command -a "systemctl status nginx"
```

```
ansible web -i inventory.ini -m uri -a "url=http://localhost"
```

systemctl status nginx:

- confirma el estado del servicio

uri:

- prueba una respuesta **HTTP** desde el servidor.

Cómo Leer el Resultado de Ansible

Estado	Significado	Lectura rápida
ok	Ya estaba correcto	No hubo cambio
changed	Ansible modificó algo	Cambio aplicado
failed	La tarea falló	Revisar error
skipped	No se ejecutó	Condición no cumplida
unreachable	No conectó al host	Revisar SSH/inventario

La segunda ejecución de un buen playbook debería mostrar menos changed.

Idempotencia

Idempotencia: la Idea Clave

- Si Nginx ya está instalado, Ansible no lo reinstala.
- Si el servicio ya está activo, no lo reinicia sin motivo.
- Si el archivo no cambió, no vuelve a copiarlo.

¿Qué significa?

Ejecutar el mismo playbook varias veces debe producir el mismo estado final, sin repetir cambios innecesarios.

```
jimmy@ubuntu-servidor:~$ ansible-playbook --ask-vault-pass --extra-vars '@password' my.yml
Vault password:

PLAY [N47] *****

TASK [Gathering Facts] *****
ok: [192.168.1.47]

TASK [installing needrestart] *****
changed: [192.168.1.47]

PLAY RECAP *****
192.168.1.47 : ok=2 changed=1 unreachable=0 failed=0

jimmy@ubuntu-servidor:~$ nano my.yml
jimmy@ubuntu-servidor:~$ ansible-playbook --ask-vault-pass --extra-vars '@password' my.yml
Vault password:

PLAY [N47] *****

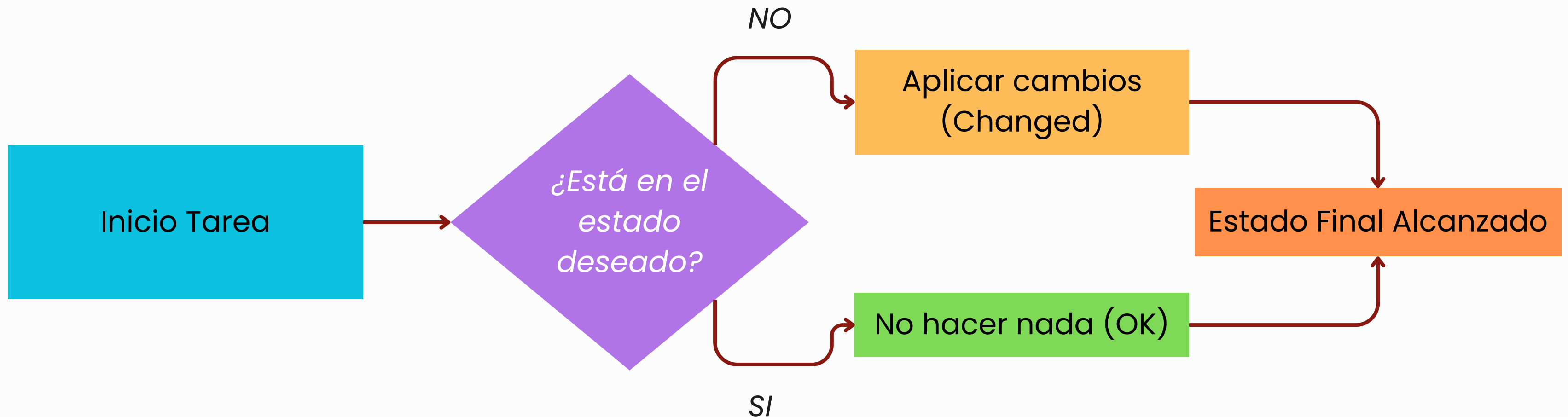
TASK [Gathering Facts] *****
ok: [192.168.1.47]

TASK [installing needrestart] *****
ok: [192.168.1.47]

PLAY RECAP *****
192.168.1.47 : ok=2 changed=0 unreachable=0 failed=0

jimmy@ubuntu-servidor:~$
```


Concepto visual



La idempotencia es la propiedad de realizar una operación varias veces sin cambiar el resultado más allá de la aplicación inicial.

Errores Clásicos y troubleshooting basico

Visibilidad Detallada:

Niveles de Detalle (-v a -vvvv)

Permite incrementar el nivel de verbosidad para observar los parámetros enviados por SSH, las respuestas de los módulos y fallos de tareas.

Diagnóstico Preciso

Esencial cuando una tarea falla de manera silenciosa o devuelve un error inesperado del sistema remoto.

```
# Normal (solo resultados)
ansible-playbook site.yml
```

```
# Verbose (-v): Muestra el resultado de cada tarea
ansible-playbook site.yml -v
```

```
# Más verbose (-vv): Incluye detalles de conexión
ansible-playbook site.yml -vv
```

```
# Muy verbose (-vvv): Incluye comandos SSH completos
ansible-playbook site.yml -vvv
```

```
# Máximo verbose (-vvvv): Incluye debug de conexión SSH
ansible-playbook site.yml -vvvv
```


Modo check (dry run) y diff

--syntax-check (Validación Estructural)

Revisa que el archivo cumpla con la sintaxis YAML y la estructura de Ansible antes de conectar con cualquier servidor remoto.

```
ansible-playbook playbook.yml --syntax-check
```

--check (Modo Dry-Run / Simulación)

Ejecuta el playbook simulando los cambios sin modificar el estado real de los servidores de destino.

```
ansible-playbook playbook.yml --check
```

--diff (Ver que cambia)

Ejecuta el playbook mostrando los cambios realizados a los servidores de destino.

```
ansible-playbook playbook.yml --diff
```

Ver qué cambiará (sin aplicar)
`ansible-playbook site.yml --check --diff`

Errores comunes y sus soluciones

Error 1: "Unreachable" – no se puede conectar

```
fatal: [ubuntu-node1]: UNREACHABLE! => {  
  "msg": "Failed to connect to the host via ssh"  
}
```

Ejemplo de Error Típico

"En la mayoría de casos es error no declarar el usuario ssh a usar"

¿Puedes hacer SSH manualmente?

```
ssh ansible@ubuntu-node1
```

¿La clave SSH es correcta?

```
ssh ansible@ubuntu-node1
```

¿El puerto SSH es el estándar?

```
ssh ansible@ubuntu-node1
```

¿El host es correcto?

```
ansible -i inventario.yml target1 -m ping -vvvv
```

Errores comunes y sus soluciones

Error 2: "Permission denied" – falta sudo

```
fatal: [target1]: FAILED! => {  
  "msg": "Missing sudo password"  
}
```

Ejemplo de Error Típico

“En la mayoría de casos es error no declarar la contraseña en el inventario o habilitar el sudo sin contraseña”

Pedir contraseña sudo

```
ansible-playbook site.yml --ask-become-pass
```

Configurar sudo sin contraseña (en el servidor)

```
echo "deploy ALL=(ALL) NOPASSWD:ALL" | sudo tee /etc/sudoers.d/deploy
```

Errores comunes y sus soluciones

Error 5: Indentación YAML incorrecta

```
ERROR! Syntax Error while loading YAML.  
mapping values are not allowed in this  
context
```

Ejemplo de Error Típico
*“Clasico error de redaccion o
mala tabulacion”*

Validar sintaxis YAML

```
python -c "import yaml; yaml.safe_load(open('playbook.yml'))"
```

Usar yamllint para más detalle

```
pip install yamllint  
yamllint playbook.yml
```

Verificar sintaxis del playbook

```
ansible-playbook --syntax-check playbook.yml
```

Errores Básicos y Cómo Corregirlos

Error común	Causa probable	Qué revisar
UNREACHABLE	SSH o inventario	IP, usuario, llave
Error de YAML	Indentación incorrecta	Espacios y dos puntos
Permiso denegado	Falta privilegio	become: true
Paquete no encontrado	Caché desactualizada	update_cache
Servicio no inicia	Configuración inválida	Logs de Nginx

La segunda ejecución de un buen playbook debería mostrar menos changed.

Gracias!!!

